RESUMEN DE PAPER · GENERACIÓN PROCEDURAL DE NIVELES 2D

Generación de niveles de Mario a partir de mecánicas mediante “cosido” de escenas

Mario Level Generation From Mechanics Using Scene Stitching

Autores	Michael Cerny Green*, Luvneesh Mugrai*, Ahmed Khalifa, Julian Togelius (New York University) · *contribución equivalente

Publicación	FDG '20: Foundations of Digital Games, 15–18 sept. 2020, Malta · preprint arXiv:2002.02992
Palabras clave	Cosido (*stitching*), diseño de niveles, algoritmos evolutivos, satisfacción de restricciones, Mario, escenas
Técnica central	Algoritmo genético FI-2Pop que ordena escenas pre-generadas para reproducir una secuencia objetivo de mecánicas

Resumen

El paper presenta un método para generar niveles de *Super Mario* “cosiendo” (*stitching*) **escenas** pre-generadas, es decir, mini-niveles que contienen mecánicas específicas. Como especificación de entrada usa la **secuencia de mecánicas** que un agente dispara al jugar un nivel original. Con un algoritmo evolutivo **FI-2Pop** y un corpus de escenas, el sistema produce niveles cuya secuencia de mecánicas se parece a la objetivo, pero con una experiencia de juego distinta. Se compara con un método **voraz** (*greedy*) que elige escenas maximizando las mecánicas objetivo: el sistema propuesto maximiza las mecánicas coincidentes y a la vez reduce las mecánicas “emergentes” no deseadas.

1. Introducción

La generación procedural de contenido (PCG) crea contenido de juegos de forma algorítmica, con enfoques muy variados: reglas escritas por el desarrollador (satisfacción de restricciones), optimización de funciones objetivo (algoritmos evolutivos), aprendizaje supervisado o por refuerzo. Dentro de la **PCG orientada a la experiencia** (*Experience-Driven PCG*), que busca provocar experiencias concretas (dificultad, estilo de juego, habilidad de un jugador), este trabajo intenta recrear la misma **“experiencia de mecánicas”** de una partida anterior, pero en niveles estructuralmente diferentes.

Evolucionar niveles completos personalizados no es trivial, así que se propone una **evolución en varios pasos**: en un trabajo previo se evolucionaron escenas que muestran subconjuntos de mecánicas (con Constrained MAP-Elites) y se publicaron como biblioteca; aquí, un optimizador FI-2Pop busca la mejor **secuencia de escenas** para igualar exactamente la traza de mecánicas de un jugador. Como las escenas se evolucionaron buscando simplicidad (baja entropía de tiles), los niveles resultantes tienen un diseño minimalista.

Aplicaciones motivadoras: entrenar indefinidamente secuencias de mecánicas en las que un jugador falla sin repetir el mismo nivel, o jugar niveles distintos pero mecánicamente análogos a uno que gustó.

2. Antecedentes

2.1 Generación procedural basada en búsqueda

La PCG basada en búsqueda usa métodos de búsqueda (a menudo algoritmos evolutivos) para encontrar buen contenido. Se ha aplicado en GVGAI, PuzzleScript, puzles de distinta dificultad, cuevas con autómatas celulares y *Cut the Rope*. El trabajo se inspira en investigaciones que separan la generación en problemas de **micro y macro evolución** para niveles grandes.

2.2 Mario AI Framework

Infinite Mario Bros. es un clon de dominio público de *Super Mario Bros.*: Mario avanza hacia una bandera a la derecha caminando, corriendo y saltando; según los hongos consumidos aguanta hasta dos golpes y puede lanzar bolas de fuego; pierde si cae a un hueco. El **Mario AI Framework** se construyó sobre él para investigación en IA: soporta agentes, generadores y muchos niveles (incluidos los originales), y se ha usado en competencias de juego y de generación.

2.3 Generación de niveles en Mario AI

Existen numerosos generadores en el framework (muchos de las competencias de 2010 y 2012):

- **Notch / Parameterized-Notch**: agregan elementos por probabilidad y verifican jugabilidad.
- **Hopper**: adapta el contenido al desempeño previo del jugador.
- **Launchpad**: gramáticas con restricciones rítmicas.
- **Occupancy-Regulated Extension**: también cose plantillas pequeñas, pero *hechas a mano*; aquí las escenas son *generadas*.
- **Pattern-based**: evoluciona niveles a partir de rebanadas verticales del juego original.
- **Grammatical Evolution**: maximiza elementos y minimiza conflictos de ubicación.

En personalización destacan *Refraction* (programación de conjuntos de respuestas), los niveles *bullet hell* con Constrained MAP-Elites y agentes que imitan estilos humanos, y en Mario los trabajos de Green et al. y Khalifa et al. sobre **escenas** que obligan a usar mecánicas concretas; el corpus de este último es la biblioteca usada aquí.

3. Métodos

Se habla de **“cosido de niveles”**: unir mini-niveles pre-evolucionados (escenas) para formar un nivel largo. Una **mecánica** es un evento que dispara el jugador (u otro objeto) y cambia el estado del juego. La Tabla 1 lista las que el sistema puede rastrear; varias pueden ocurrir a la vez (saltar y golpear un bloque). Hay muchos tipos de salto porque saltar es el núcleo del juego.

Tabla 1. Mecánicas del juego que pueden registrarse durante una partida.

Nombre	Descripción
Jump	Mario dejó el suelo.
Low Jump	Salto bajo (cambio pequeño en el eje y, bajo un umbral).
High Jump	Salto alto (cambio grande en el eje y, sobre un umbral).
Short Jump	Salto corto (cambio pequeño en el eje x).
Long Jump	Salto largo (cambio grande en el eje x).
Stomp Kill	Mario mata a un enemigo saltándole encima.
Shell Kill	Mario mata a un enemigo empujando un caparazón de Koopa.
Fall Kill	Un enemigo muere al caer fuera de la pantalla.
Mode	Mario cambia de modo (pequeño, grande, fuego).
Coin	Mario recoge una moneda.
Brick Block	Mario golpea un bloque de ladrillo.
Question Block	Mario golpea un bloque de interrogación.

Una **escena** es una zona pequeña del nivel que encapsula una idea (Fig. 1). Aquí representa el subconjunto de mecánicas que se ejercitan en ella. La biblioteca debería cubrir todas las combinaciones, desde escenas planas sin mecánicas hasta escenas con todas.

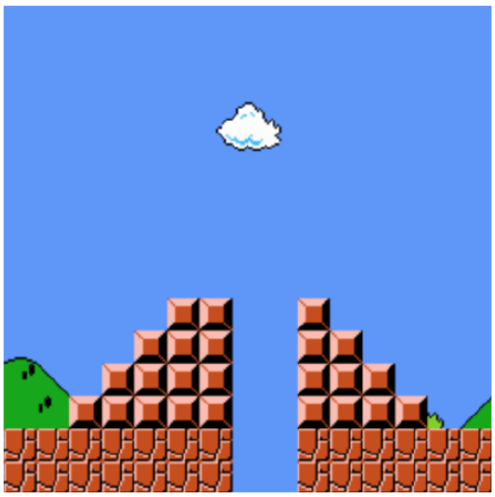


Figura 1. Escena de Super Mario Bros. en la que Mario debe saltar de la primera pirámide a la segunda para superarla.

El nivel completo se evoluciona con **FI-2Pop** (*Feasible-Infeasible Two-Population*): una población infactible intenta satisfacer restricciones (jugabilidad) y una factible optimiza la aptitud (parecido de mecánicas). Los cromosomas migran entre poblaciones según cumplan o no las restricciones, y se usa elitismo entre generaciones.

3.1 Representación del cromosoma

Un nivel es una secuencia de longitud variable de escenas cosidas; el cromosoma *es* el nivel. Una escena puede estar etiquetada con varias mecánicas, lo que permite niveles más condensados.

3.2 Operadores genéticos

Cruce de dos puntos con longitud variable: se eligen dos puntos en cada padre y se intercambia el tramo central, lo que puede alargar o acortar el nivel e intercambiar desde una escena hasta el nivel entero. **Mutación**, con cinco tipos:

- **Delete**: borrar una escena.
- **Add**: agregar una escena aleatoria adyacente (selección por ranking, con probabilidad inversa al número de mecánicas).
- **Split**: dividir una escena en dos, repartiendo al azar sus mecánicas entre ambas.
- **Merge**: juntar las mecánicas de una escena con su vecina y reemplazar ambas por una escena del corpus con la lista combinada.
- **Change**: cambiar una escena por otra aleatoria (misma selección inversa por ranking).

La escena a mutar se elige favoreciendo las que tienen más mecánicas.

3.3 Restricciones y cálculo de aptitud

Restricción. Se ejecuta un agente artificial (A*) N veces sobre el nivel:

```
C = (1/N) · Σ d_i / d_level     si (1/N) · Σ w_i <  p
C = 1                           si (1/N) · Σ w_i ≥  p            (1)
```

donde d_i es la distancia recorrida por el agente en la corrida i, d_{level} la longitud máxima del nivel, $w_i = 1$ si el agente llegó al final y p el porcentaje umbral de victorias.

Aptitud. Se toma la partida en que el agente ganó disparando la menor cantidad de mecánicas. Se parte de un puntaje inicial S y se descuenta por **fallas**: una mecánica **faltante** (que habría completado una subsecuencia correcta) o una mecánica **extra** intercalada (Fig. 2).

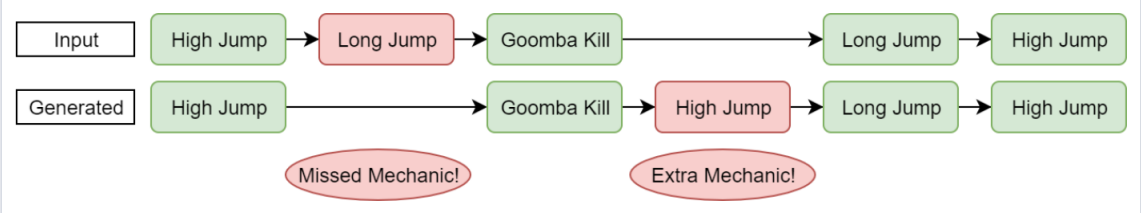


Figura 2. Fallas de mecánica: el *Long Jump* de la entrada falta en la partida generada (mecánica perdida), y la partida generada agrega un *High Jump* que no estaba (mecánica extra).

El conteo de fallas usa un algoritmo de emparejamiento de secuencias (Algoritmo 1): recorre la secuencia objetivo y busca la primera aparición de cada mecánica en el resto de la secuencia generada; si no aparece, suma una faltante; si aparece en el índice k, suma k extras y avanza el puntero.

```
Algoritmo 1 (conteo de fallas) — versión resumida
pGen = 0; extra = 0; missed = 0
for target in targetSeq:
    k = indexOf(target, generatedSeq[pGen:])
    if k == -1:  missed += 1
    else:        extra += k;  pGen += k + 1
    if pGen >= len(generatedSeq): break
return extra, missed
```

Nota del resumen: en el pseudocódigo publicado, el avance del puntero aparece escrito sobre *pTarget*; por la descripción del texto, el puntero que debe avanzar es el de la secuencia generada, como se muestra arriba.

La aptitud (que puede resultar negativa) es:

```
P_missed = W_missed · M_missed
P_extra  = a · tanh(b · M_extra) + c                          (2)
F        = S − (P_missed + P_extra · (S − P_missed))
```

con M_{missed} y M_{extra} el número de mecánicas faltantes y extra, y a, b, c, W_{missed} pesos predefinidos. Las faltantes penalizan linealmente; las extras penalizan de forma saturada (tanh) y proporcional a lo que queda del puntaje.

4. Experimentos

Se usan como objetivo tres niveles originales de Super Mario Bros.: **1-1**, **4-2** y **6-1**. El agente A* de Robin Baumgarten (ganador de la primera Mario AI Competition) juega cada uno una vez y su secuencia de mecánicas se usa como objetivo (Tabla 2).

Tabla 2. Métricas de las trazas objetivo: total de mecánicas, mecánicas únicas, la más y la menos frecuente. “Jump” domina.

Nivel	Total	Únicas	Más frecuente	Menos frecuente
1-1	57	8	Jump (18)	Stomp Kill (1)
4-2	96	8	Jump (29)	Stomp Kill (2)
6-1	66	6	Jump (22)	Coin (1)

Configuración del algoritmo evolutivo: población de 250, cruce 70 %, mutación 20 %, 1 élite. Cada cromosoma se inicializa con entre 5 y 25 escenas; el número de mecánicas de cada escena se muestrea de una gaussiana con media igual al promedio de mecánicas del objetivo y desviación 1. El corpus (del trabajo *Intentional Computational Level Design*) tiene **1691 escenas** con casi todas las combinaciones posibles de mecánicas.

Líneas base: (1) **aleatoria**: 5–25 escenas elegidas al azar; (2) **voraz**: 5–25 escenas elegidas para maximizar las mecánicas coincidentes según las etiquetas del corpus. Para ambas se generan niveles hasta obtener **20 niveles jugables** por mundo (que el A* complete).

5. Resultados

La Fig. 3 compara los originales con sus versiones voraces y evolucionadas. Ambos métodos producen niveles con menos variedad gráfica y más planos, porque el corpus fue evolucionado con una función de entropía que presiona hacia espacios uniformes y baja variación de altura. Para forzar secuencias de mecánicas, los niveles evolucionados llegan al extremo de usar muros verticales altos de 1 tile de ancho que obligan a varios saltos seguidos.

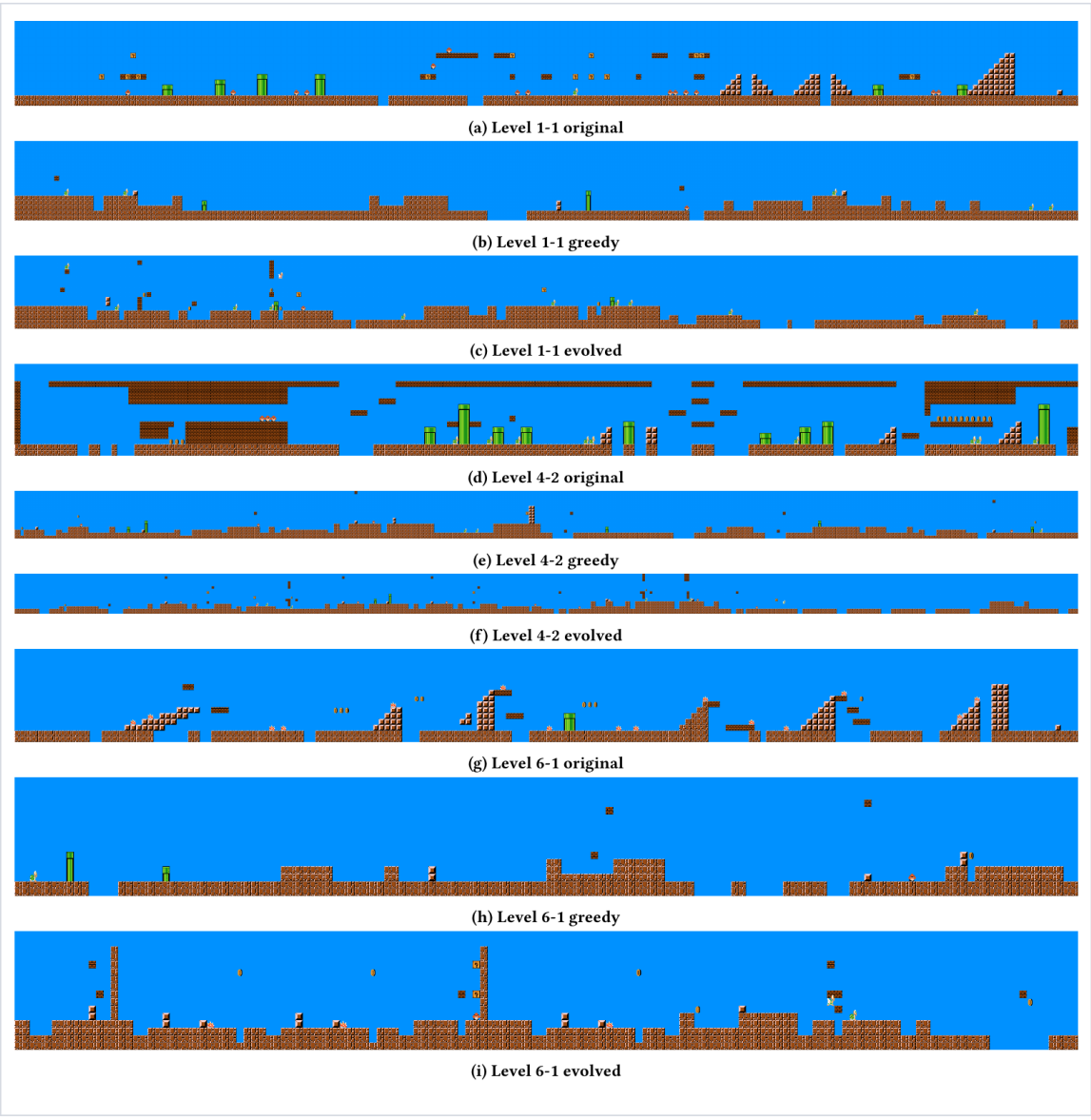


Figura 3. Muestra aleatoria de niveles voraces y evolucionados frente a sus originales: (a–c) 1-1 original / voraz / evolucionado; (d–f) 4-2; (g–i) 6-1.

5.1 Desempeño de la evolución

La Fig. 4 muestra la aptitud normalizada promedio (20 niveles por objetivo). En **1-1** la mejora es casi lineal hasta la generación ~80 y luego se desacelera (secuencia fácil de igualar). **6-1** sube rápido en las primeras 20 generaciones, con mayor desviación. **4-2** crece rápido hasta ~40 y luego pasa a ganancias sublineales, con desviación pequeña.

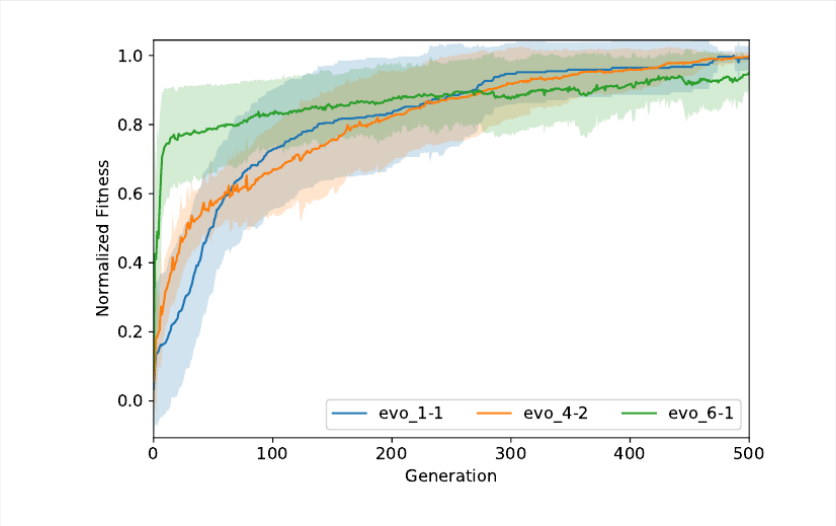


Figura 4. Aptitud normalizada a lo largo de las generaciones para los tres niveles objetivo (área sombreada = desviación estándar).

5.2 Jugabilidad

Se corre el A* 20 veces por nivel y se promedia (Tabla 3). El A* gana solo el **52 %** de los niveles originales (excluyendo subacuáticos y castillos), lo que muestra que no es perfecto. Los evolucionados están cerca del **100 %**; los voraces rondan el 25–28,5 % y los aleatorios ~10 %.

Tabla 3. Porcentaje de victorias del A* de Robin Baumgarten sobre cada conjunto (20 corridas por nivel).

Experimento	Jugabilidad
Niveles originales	52 %
Niveles aleatorios	10,75 %
Voraz – Mundo 1-1	28,5 %
Voraz – Mundo 4-2	26,25 %
Voraz – Mundo 6-1	25 %
Evolución – Mundo 1-1	**100 %**
Evolución – Mundo 4-2	**99,5 %**
Evolución – Mundo 6-1	**87,25 %**

5.3 Similitud de mecánicas

La Fig. 5 muestra coincidencias (*Matches*) y extras (*Extras*) normalizadas por el total de la Tabla 2. En los tres niveles la evolución supera ampliamente a voraz y aleatorio en coincidencias, y minimiza extras en 1-1 y 6-1. En **4-2**, tras una caída inicial, la evolución vuelve a agregar extras hasta la generación ~50 (probablemente inevitable para aumentar coincidencias); ambas curvas se estabilizan hacia la generación 200.

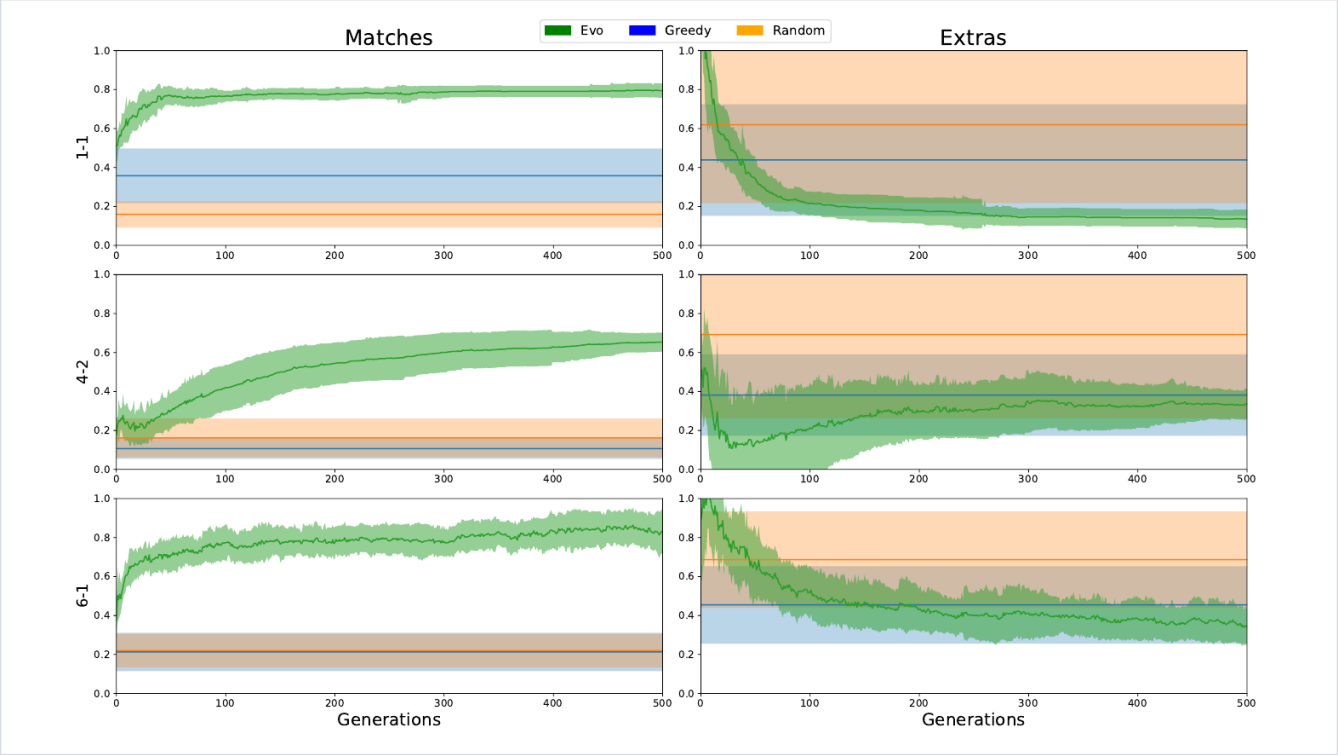


Figura 5. Estadísticas de mecánicas por generación (filas: 1-1, 4-2, 6-1; columnas: coincidencias y extras). Verde: evolución; azul: voraz; naranja: aleatorio.

5.4 Diversidad estructural

Se usa la **Divergencia KL de Patrones de Tiles** (TPKLDiv) con ventana de 3×3. Para cada nivel se toma la mínima TPKLDiv frente a los otros 19 (peor caso) y se promedia (Tabla 4). Todas las técnicas son menos diversas que los originales; incluso los aleatorios, lo que indica que las escenas en sí son poco diversas. Los niveles evolucionados para 1-1 tienen poca diversidad (es el nivel más simple), y sorprende que 4-2 tenga aún menos: con tantas mecánicas cuesta encontrar niveles jugables variados. Los voraces son más diversos que los evolucionados, pero mucho menos jugables.

Tabla 4. TPKLDiv entre los niveles generados de cada tipo (media ± desv. estándar); los originales como referencia.

Experimento	TPKLDiv
Niveles originales	0,715 ± 0,410
Niveles aleatorios	0,697 ± 0,265
Voraz – Mundo 1-1	0,675 ± 0,228
Voraz – Mundo 4-2	0,648 ± 0,181
Voraz – Mundo 6-1	0,648 ± 0,172
Evolución – Mundo 1-1	0,269 ± 0,127
Evolución – Mundo 4-2	0,264 ± 0,094
Evolución – Mundo 6-1	0,348 ± 0,117

También se midió la TPKLDiv de cada nivel generado frente a su original (Tabla 5). Los aleatorios son los más distintos, salvo en 4-2, donde el voraz lo supera (posiblemente por generar niveles más largos). Los evolucionados tienen cerca de la mitad de la divergencia de los aleatorios, excepto en 4-2: estos niveles son poco diversos entre sí, pero más distintos de su original.

Tabla 5. TPKLDiv promedio entre cada nivel generado y su nivel original (los aleatorios se comparan con cada objetivo como referencia).

Experimento	TPKLDiv
Aleatorio – 1-1	2,941 ± 1,005
Voraz – Mundo 1-1	2,636 ± 0,795
Evolución – Mundo 1-1	1,601 ± 0,573
Aleatorio – 4-2	2,942 ± 1,005
Voraz – Mundo 4-2	3,329 ± 0,647
Evolución – Mundo 4-2	1,997 ± 0,466
Aleatorio – 6-1	2,942 ± 1,005
Voraz – Mundo 6-1	2,601 ± 0,577
Evolución – Mundo 6-1	1,505 ± 0,404

6. Discusión

El generador FI-2Pop supera claramente a las líneas base en igualar la secuencia de mecánicas. Sobre la **longitud**: un nivel típico de Mario equivale a ~14 escenas, y los niveles para 1-1 y 6-1 convergen a ese valor; los de 4-2 convergen a ~23 escenas (≈1,64×). La hipótesis: 4-2 tiene muchas más mecánicas (≈1,7× las de 1-1 y ≈1,5× las de 6-1), así que el generador "esparce" las mecánicas en más escenas para asegurar el orden; al alargarse, también aumentan las mecánicas extra (Fig. 6).

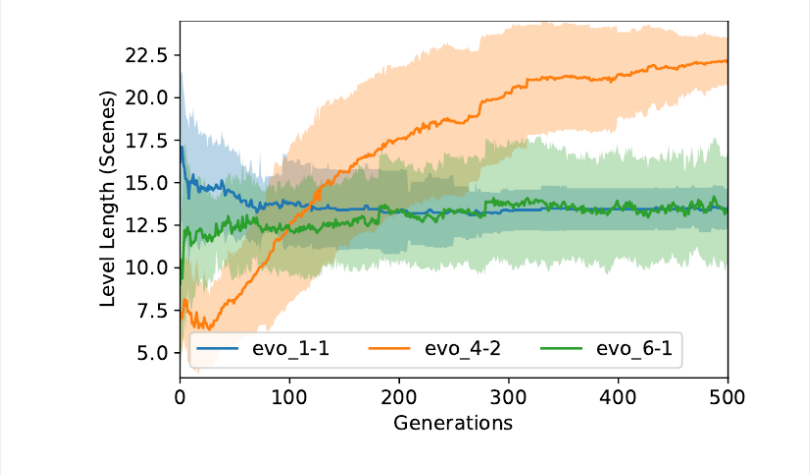


Figura 6. Longitud promedio de los niveles (en escenas) a lo largo de las generaciones para los tres objetivos.

La presión de emparejamiento hace que la evolución **fuerce** el orden de las mecánicas: el agente no tiene otra opción que ejecutarlas para avanzar. En la Fig. 7, el original 6-1 exige una secuencia de 4 saltos; el voraz solo pone escenas con saltos cerca (y un Mario grande podría chocar con el Koopa en vez de saltarlo), mientras que el evolucionado usa un muro largo como "puerta" que solo se supera tras 3 saltos para ganar altura. Es una especie de **generación minimalista**: los niveles más simples que son mecánicamente análogos al original.

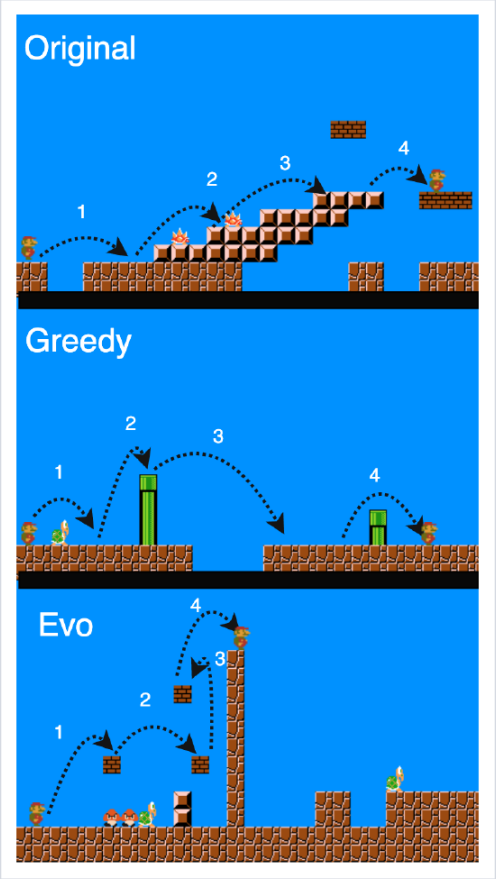


Figura 7. Una secuencia de 4 saltos en el nivel 6-1 original y cómo intentan copiarla los generadores voraz y evolutivo.

Con base en la diversidad, los autores plantean que una secuencia con pocas mecánicas simples (1-1) tiene pocos "primos" mecánicamente similares, y que muchas mecánicas (4-2) también dificultan la diversidad; un nivel intermedio podría ser el punto óptimo (el texto menciona 6-2, probablemente refiriéndose a 6-1).

7. Conclusiones

Se exploró cómo generar contenido personalizado cosiendo escenas pre-generadas para construir niveles de Super Mario, comparando la secuencia de mecánicas del A* en los niveles originales (1-1, 4-2, 6-1) con la de los niveles generados. FI-2Pop produce niveles **ganables y mecánicamente análogos** y empareja la secuencia mucho mejor que las líneas base. La menor diversidad respecto a los originales se atribuye a la presión de entropía con que se generó el corpus de escenas.

Trabajo futuro y conclusiones clave

- Usar trazas de agentes "personificados" o jugadores humanos para generar niveles a la medida, y un estudio de usuarios sobre preferencias entre original y generado.
- Regenerar el corpus con una **entropía objetivo** (p. ej. la media de los niveles originales) en lugar de minimizarla, ya que los niveles de Mario nunca fueron minimalistas.
- **Fortaleza:** jugabilidad cercana al 100 % y alta fidelidad mecánica frente a voraz/aleatorio.
- **Limitación:** baja diversidad estructural y estética (niveles planos, muros de 1 tile).